

Video-Skript & Protokoll

Folge 3: Autonomer Code Generator & Visual Validator (qwen2.5v1)

Kanal: Data, Paper & Code

30. September 2026

Inhaltsverzeichnis

1	Struktur & Thematische Gliederung	2
2	YouTube Sprecherskript (Vortragstext)	2
3	Theoretischer Hintergrund	3
3.1	Warum autokorrektive Agenten-Schleifen?	3
3.2	Architektur des Agenten-Loops	4
4	System-Setup & Voraussetzungen	4
5	Vollstaendiges Python-Skript des Agenten	4
6	Ergebnis & Agenten-Protokoll	8
7	Ausblick & Anwendungsbereiche	8

1 Struktur & Thematische Gliederung

Nr.	Themenblock	Kerninhalt & Zielsetzung
01	Begrüßung & Hook	Die Grenzen blindsichtiger Coder-LLMs.
02	Das Agenten-Prinzip	Closed-Loop Feedback: Generieren, Ausführen, Visuell Validieren.
03	Subprocess Execution	Sichere Ausführung von LLM-Code in isolierten Laufzeiten.
04	Vision-Feedback Loop	Wie <code>qwen2.5v1</code> Diagramme liest und Freigaben (APPROVED) erteilt.
05	Live-Demo & Ausführung	Schritt-für-Schritt-Durchlauf des Python-Agenten.
06	Fazit & Ausblick	Einsatzmöglichkeiten für automatisierte Berichte & Dashboards.

Tabelle 1: Ablaufplan der dritten Videofolge

2 YouTube Sprecherskript (Vortragstext)

Teil 1: Der Hook — Wenn LLMs blind Code schreiben

[Kamera: Nahaufnahme | Dynamisch & direkt]

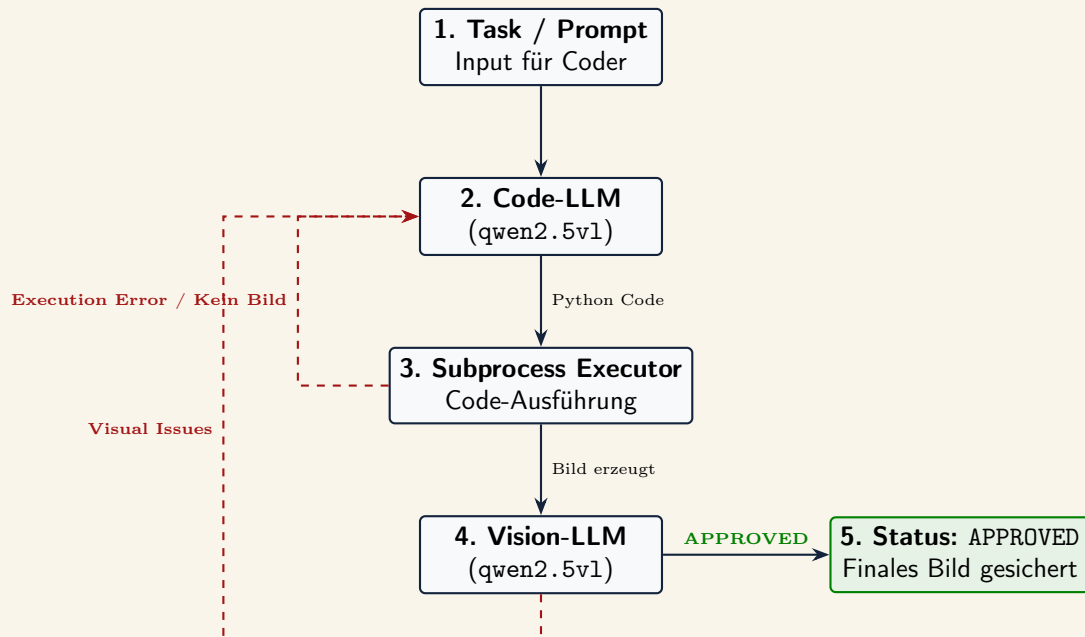
„Stellt euch vor, ihr bittet ein Sprachmodell, euch ein schönes Diagramm zu zeichnen. Das Modell liefert brav den Python-Code, ihr führt ihn aus — und BUMM: Entweder stürzt das Skript mit einem dicken Traceback ab, oder die Grafik sieht furchtbar aus. Überlappende Achsen, fehlende Titel, abgeschnittene Daten.

Das Problem? Klassische Coder-LLMs arbeiten völlig ****blind****. Sie wissen nicht, wie ihr erzeugter Code visuell aussieht.

Schluss damit. Willkommen bei **Data, Paper & Code**! Heute bauen wir einen ****vollständig autonomen, lokalen Software-Agenten****. Er schreibt Python-Code, führt ihn lokal aus, faengt Fehler ab und nutzt das lokale Vision-Modell `qwen2.5v1` über Ollama, um das erzeugte Diagramm visuell zu prüfen. Erst wenn das Vision-Modell **APPROVED** sagt, ist der Job erledigt. Komplett lokal, kostenlos und ohne Cloud-APIs!“

Teil 2: Das Agenten-Pattern (Closed-Loop Loop)

[Bildschirm: Architektur-Diagramm des Iterations-Ablaufs]



„Unser Agent nutzt ein schlaues Schleifen-Prinzip: 1. **Der Codier:** `qwen2.5v1` generiert reinen Python-Code basierend auf unserer Aufgabenstellung. 2. **Der Executor:** Ein Python-subprocess führt den Code aus. Tritt ein Syntax- oder Laufzeitfehler auf, geht die Fehlermeldung direkt zurück an den Codier zur Korrektur. 3. **Der Validator:** Erzeugt das Skript erfolgreich ein Bild, schlägt die Stunde des Vision-LLMs. Es analysiert das Bild auf Lesbarkeit, Achsenbeschriftungen und Layout. Gibt das Vision-Modell grünes Licht mit **APPROVED**, stoppt die Schleife. Wenn nicht, wandert die Kritik zur nächsten Überarbeitung!“

Teil 3: Live-Demo & Der perfekte Durchlauf

[Bildschirm: VS Code / Terminal geöffnet]

„Lassen wir unseren Agenten live arbeiten! Unsere Aufgabe an den Agenten: Lade Börsendaten via `yfinance` für den ETF `VUSD.L` herunter und erstelle ein sauber formatiertes Chart. Schaut euch die Konsole an: In **Iteration 1** generiert der Codier den Python-Code. Der Subprocess führt ihn aus und erzeugt die Datei `Kurve_iter_1.jpg`. Jetzt schickt unser Agent das erzeugte Bild an Ollama. `qwen2.5v1` liest das Bild, prüft Achsen, Gitter und Titel... und BUMM! Das Vision-Modell antwortet direkt mit **APPROVED**. Der Agent speichert das finale Ergebnis als `Kurve_final.jpg` ab und beendet die Aufgabe erfolgreich beim allerersten Versuch!“

3 Theoretischer Hintergrund

3.1 Warum autokorrektive Agenten-Schleifen?

Klassische Code-Generierung leidet unter dem ***Open-Loop-Problem***: Fehler werden erst nach der manuelle Ausführung durch den Entwickler entdeckt. Ein autokorrektiver Agent schliesst diese Lücke durch automatisiertes Traceback-Parsing und visuelles Feedback.

3.2 Architektur des Agenten-Loops

Der Prozess teilt sich in drei funktionale Komponenten auf:

1. **Code Generation Phase:** Generierung von isoliertem Python-Code ohne Markdown-Formatiertag, gesteuert über ChatML-System-Prompts.
2. **Subprocess Sandbox Execution:** Ausführung des erzeugten Skripts über den Python-Interpreter mittels `subprocess.run`. Laufzeitfehler werden über `stderr` erfasst.
3. **Multimodal Visual Validation:** Einbettung des Ergebnisses als Base64-String in die Ollama-API. Ein speziell strukturierter Prompt fordert ein binäres Signal (**APPROVED** vs. Aenderungswunsch).

4 System-Setup & Voraussetzungen

Anleitung: Agenten-Umgebung vorbereiten

1. **Ollama bereitstellen:** Stelle sicher, dass Ollama im Hintergrund laeuft.
2. **Modell bereitstellen:** Lade das multimodale Modell einmalig herunter:

```
ollama run qwen2.5vl
```

3. **Python-Pakete installieren:**

```
pip install requests yfinance matplotlib pillow
```

5 Vollstaendiges Python-Skript des Agenten

Das folgende Skript steuert den vollständigen autonomen Verifikations- und Korrektur-Prozess.

```
1 import base64
2 import os
3 import subprocess
4 import sys
5 import requests
6
7 OLLAMA_GENERATE_URL = "http://localhost:11434/api/generate"
8 CODER_MODEL = "qwen2.5vl"
9 VISION_MODEL = "qwen2.5vl"
10
11 SCRIPT_FILE = "generated_chart_script.py"
12 TEMPLATE_IMAGE = "Kurve_Vorlage.jpg"
13 BASE_IMAGE_NAME = "Kurve"
14 MAX_ATTEMPTS = 4
15
16
17 def encode_image_to_base64(image_path: str) -> str:
18     """Wandelt ein lokales Bild in einen Base64-String um."""
19     with open(image_path, "rb") as image_file:
```

```

20         return base64.b64encode(image_file.read()).decode('utf-8')
21
22
23 def analyze_chart_with_ollama(
24     image_path: str,
25     prompt_text: str,
26     ollama_url: str = OLLAMA_GENERATE_URL,
27     model_name: str = VISION_MODEL
28 ) -> str:
29     """Sende das Bild an das Vision-Modell in Ollama."""
30     if not os.path.exists(image_path):
31         return "[FEHLER] Das Bild wurde nicht erstellt."
32
33     base64_image = encode_image_to_base64(image_path)
34
35     payload = {
36         "model": model_name,
37         "prompt": prompt_text,
38         "images": [base64_image],
39         "stream": False,
40         "options": {"temperature": 0.2}
41     }
42
43     try:
44         response = requests.post(ollama_url, json=payload, timeout=120)
45         response.raise_for_status()
46         return response.json().get("response", "Keine Antwort vom Modell erhalten.")
47     except requests.exceptions.RequestException as e:
48         return f"[FEHLER]: Anfrage an Ollama fehlgeschlagen: {e}"
49
50
51 def run_code_llm(prompt: str) -> str:
52     """Ruft das Coder-Modell ueber Ollama ab."""
53     full_prompt = (
54         "<|im_start|>system\n"
55         "You are an expert Python developer. Write complete, valid Python code.\n"
56         "Do NOT use markdown backticks (no ``python). Return ONLY the executable Python code.<|im_end|>\n"
57         f"<|im_start|>user\n{prompt}<|im_end|>\n"
58         "<|im_start|>assistant\n"
59     )
60
61     payload = {
62         "model": CODER_MODEL,
63         "prompt": full_prompt,
64         "stream": False,
65         "options": {
66             "temperature": 0.2,
67             "num_predict": 1024
68         }
69     }
70
71     try:
72         response = requests.post(OLLAMA_GENERATE_URL, json=payload, timeout=90)
73         response.raise_for_status()
74

```

```

75     result_json = response.json()
76     code_output = result_json.get("response", "").strip()
77
78     if not code_output:
79         payload["prompt"] = prompt
80         res = requests.post(OLLAMA_GENERATE_URL, json=payload, timeout
=90).json()
81         code_output = res.get("response", "").strip()
82
83     return code_output
84
85     except Exception as e:
86         print(f"      Fehler bei Anfrage an Coder-Modell ({CODER_MODEL}): {e
}")
87         return ""
88
89
90 def run_agent(task_description_en: str):
91     print("=====")
92     print("AGENT: Chart Code Generator & Visual Validator")
93     print(f"Vorlage: {TEMPLATE_IMAGE}")
94     print("=====\\n")
95
96     history = ""
97
98     for attempt in range(1, MAX_ATTEMPTS + 1):
99         print(f"--- Iteration {attempt}/{MAX_ATTEMPTS} ---")
100
101         current_image_file = f"{BASE_IMAGE_NAME}_iter_{attempt}.jpg"
102
103         # 1. CODE GENERIEREN
104         prompt = (
105             f"You are a Python developer. Task: {task_description_en}\\n"
106             f"Requirements:\\n"
107             f"- Save the generated chart image strictly as '{
current_image_file}'.\\n"
108             f"- Return ONLY the raw executable Python code. No markdown
blocks, no conversational text.\\n"
109             f"{history}"
110         )
111
112         raw_code = run_code_llm(prompt)
113
114         if not raw_code or len(raw_code.strip()) == 0:
115             print("      Fehler: Das LLM hat einen leeren String
zurueckgegeben. Erneuter Versuch...")
116             history += "\\nYour last response was completely empty. You
MUST generate valid Python code!"
117             continue
118
119         # Markdown-Tags bereinigen
120         code = raw_code
121         if code.startswith("```"):
122             lines = code.split("\\n")
123             code = "\\n".join(lines[1:-1]) if lines[-1].startswith("```")
else "\\n".join(lines[1:])
124
125         with open(SCRIPT_FILE, "w", encoding="utf-8") as f:
126             f.write(code)

```

```

127
128     # 2. CODE AUSFUEHREN
129     res = subprocess.run([sys.executable, SCRIPT_FILE], capture_output
130                             =True, text=True)
131
132     if res.returncode != 0:
133         print(f"      Syntax/Runtime Error:\n{res.stderr.strip()}")
134         history += f"\nPrevious attempt failed with error:\n{res.
135             stderr}\nFix the code."
136         continue
137
138     if not os.path.exists(current_image_file):
139         print(f"      Fehler: Das Skript ist durchgelaufen, hat aber '{
140             current_image_file}' nicht erzeugt.")
141         history += f"\nThe script ran, but failed to save the image to
142             '{current_image_file}'. Make sure to call plt.savefig('{
143             current_image_file}')."
144         continue
145
146     print(f"      Skript fehlerfrei ausgefuehrt und '{current_image_file
147         }' erzeugt.")
148
149     # 3. VISION-ANALYSE
150     vision_prompt = (
151         "Analyze the generated chart picture critically:\n"
152         "1. Is the chart correctly rendered and legible?\n"
153         "2. Are axes labeled properly without overlapping text?\n"
154         "If the chart looks complete and well-formatted, reply with '
155         APPROVED'. "
156         "Otherwise, explain shortly what needs to be improved."
157     )
158
159     print(f"[INFO] Sende '{current_image_file}' zur visuellen Pruefung
160         an {VISION_MODEL}...")
161
162     evaluation = analyze_chart_with_ollama(current_image_file,
163         vision_prompt)
164     print(f"Vision Feedback:\n{evaluation}\n")
165
166     if "APPROVED" in evaluation.upper():
167         print("      ERFOLG: Das Skript erzeugt ein fehlerfreies und
168             visuell geprueftes Diagramm!\n")
169         print("=" * 50)
170         print(" INHALT VON generated_chart_script.py:")
171         print("=" * 50)
172         print(code)
173
174         final_image = f"{BASE_IMAGE_NAME}_final.jpg"
175         with open(current_image_file, "rb") as src, open(final_image,
176             "wb") as dst:
177             dst.write(src.read())
178             print(f"      Das finale Diagramm wurde als '{final_image
179                 }' abgespeichert.")
180             break
181
182         history += f"\nThe generated image had visual issues:\n{evaluation
183             }\nPlease update the code."

```

```

173 if __name__ == "__main__":
174     task = (
175         "Write a Python script using yfinance and matplotlib to download 3
176         months of data for 'VUSD.L'. "
177         "Plot the Close price with a grid and proper title."
178     )
179     run_agent(task)

```

Listing 1: Vollstaendiger Python-Code des Vision-Code-Agenten

6 Ergebnis & Agenten-Protokoll

Protokoll des erfolgreichen Agenten-Laufs

Ausfuehrung: python Software_Agent03_with_image.py

```

=====
AGENT: Chart Code Generator & Visual Validator
=====

```

-- Iteration 1/4 --

Skript fehlerfrei ausgefuehrt und 'Kurve_iter_1.jpg' erzeugt.

[INFO] Sende 'Kurve_iter_1.jpg' zur visuellen Pruefung an qwen2.5vl...

Vision Feedback:

APPROVED. The chart is correctly rendered and legible. The axes are labeled properly without overlapping text. The title and the legend are clearly visible and the data points are distinct.

Ergebnis: ERFOLG: Das Skript erzeugt ein fehlerfreies und visuell geprueftes Diagramm!

7 Ausblick & Anwendungsbereiche

Das gezeigte autokorrektive Agenten-Muster schließt die Lücke zwischen generativer KI und rigider wissenschaftlicher Datenanalyse. Es lässt sich nahtlos auf automatisierte Pipelines in Forschung und Entwicklung übertragen:

- **Wissenschaftliche Regressionsdiagnostik:** Automatische Erzeugung, visuelle Verifikation und Interpretation komplexer Prüfdiagramme (z. B. *Normalized Residual Plots* zur Prüfung von Modellannahmen).
- **Quality Assurance für Research Papers:** Automatisierte Überprüfung von generierten Abbildungen auf Lesbarkeit, korrekte Skalierung und Einhaltung von Publizierungsstandards (z. B. IEEE- oder Nature-Layouts).
- **Autonome Reports:** Dynamisches Erstellen reproduzierbarer PDF-Berichte aus Roheader-Pipelines inkl. visueller Selbstkontrolle der eingebetteten Diagramme.
- **Multimodale Benchmark-Systeme:** Automatisierte Evaluation von Open-Source-Vision-LLMs hinsichtlich ihrer Fähigkeit, technische und mathematische Darstellungen korrekt zu interpretieren.